

FoodRescue AI: Supermarket Food Waste Reduction

Team ID and Team Name: LAB-P6-Team 8

Team Members:

Jariel Seah, 2601663, 2601663@sit.singaporetech.edu.sg

Darren Lee Khai Xuan, 2603021, 2603021@sit.singaporetech.edu.sg

Ng Jiexin, Jacinth, 2604333, 2604333@sit.singaporetech.edu.sg

Quek Wei Foong, 2601766, 2601766@sit.singaporetech.edu.sg

Neo Jun Hui, 2605731, 2605731@sit.singaporetech.edu.sg

Date of submission: 14th October 2026

Table of Contents

Introduction 3

Data Flow Diagram 4

Data Structure and Storage Design 5

Exception Handling Matrix 5

Introduction

Supermarkets frequently face challenges in managing food products that are nearing their use-by dates or experiencing low customer demand. Without immediate action from supermarket staff, these products quickly turn into food waste, resulting in financial losses and environmental harm. According to the United Nations Environment Programme, food loss and waste generate an estimated 8% to 10% of global greenhouse gas emissions.

To address this problem, our team developed FoodRescue AI. This application helps supermarket staff find ways to use food before it goes to waste by suggesting recipes. Through the application, supermarket staff can manage the inventory by viewing, searching and updating records. They are also able to request AI assessments by selecting a product and entering observations about a product's condition. The AI assessments use details such as the product's use-by date, quantity, demand, storage condition and allergen information to assess and provide recipe suggestions. Each recipe includes an ingredient list and preparation instructions.

For example, the AI could suggest using tomatoes that are close to their use-by date in a pasta dish. The suggested dish would then be checked against our list of predefined recipes, with any suggestions that do not match being rejected. Additionally, products that are expired, spoiled, or deemed unsuitable for consumption are rejected, regardless of whether the ingredients match the predefined list. Accepted recipes and reasons for rejection are saved for future reference. However, the staff are still responsible for checking food safety before preparing and selling any suggested dishes with the ingredients.

Through FoodRescue AI, we aim to reduce avoidable food waste, improve inventory management efficiency, and help supermarkets maximise the use of available ingredients while maintaining food safety standards.

Data Flow Diagram

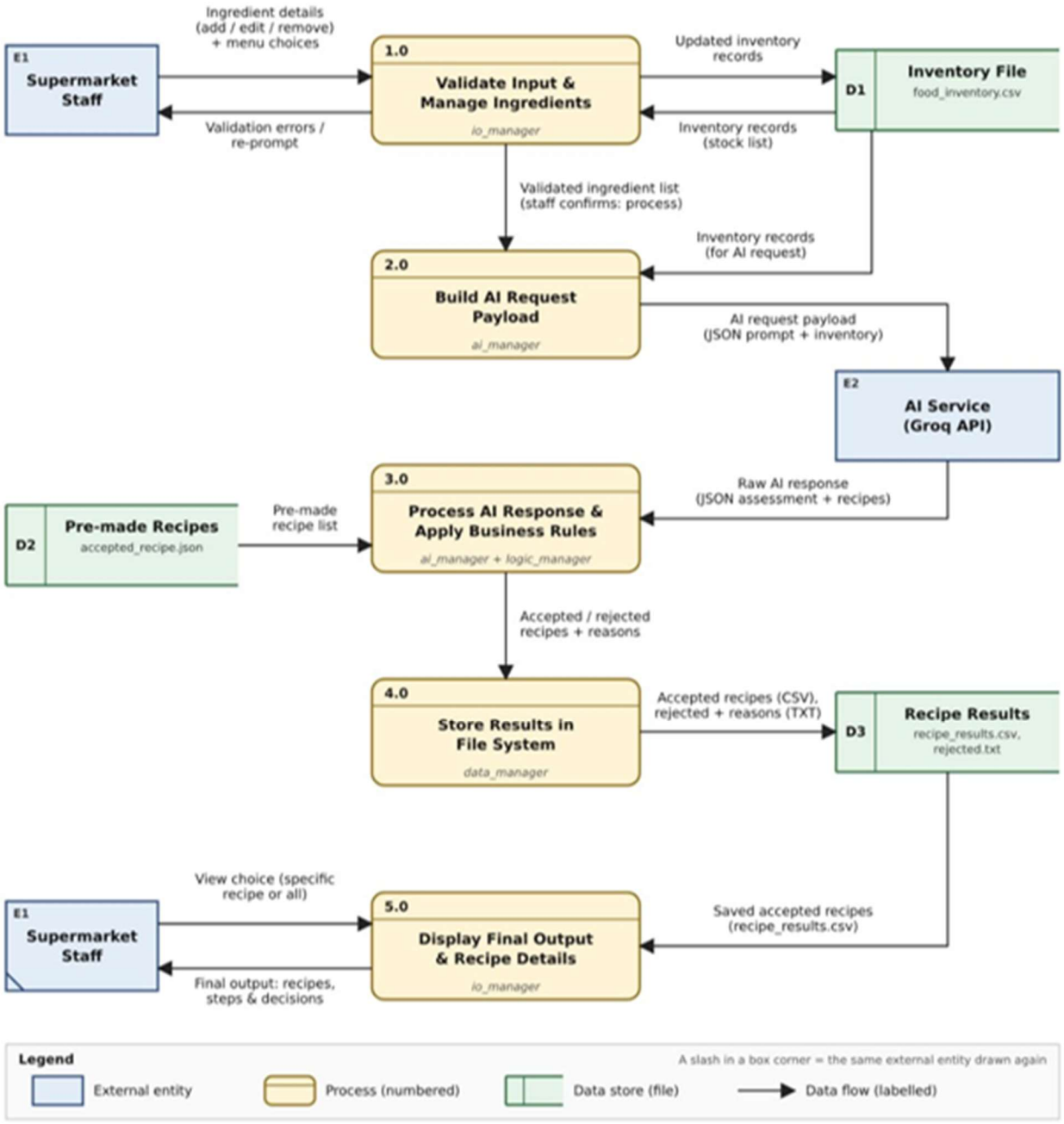


Figure 1: Data Flow Diagram

Data Structure and Storage Design

FoodRescue AI stores its data in three different file types. A CSV file holds the supermarket's food inventory, with fields such as category, use-by date, storage condition, demand level and quantity. JSON files hold the recipe data. recipes.json contains the AI-generated recipes and accepted_recipe.json contains the supermarket's fixed list of pre-made recipes. Each AI recipe is checked against that list, and approved recipes will be saved to a CSV file (recipe_results.csv). Whereas rejected recipes will be saved to a TXT file (rejected.txt) together with reason for rejection.

Files used	Purpose
food_inventory.csv	Tracks food items by category, use-by dates, days to expiry, storage condition, waste risk, allergens, and demand levels for AI manager to check through
recipes.json	Contains AI-recommended recipes based on selected ingredient, saved with food name, required ingredients and the preparation instructions
accepted_recipe.json	Holds a pre-made recipe list used by logic manager to evaluate the AI-generated recipes and decide whether to approve or reject it based on criteria such as ingredients and recipe name.
recipe_results.csv	Stores approved recipes for supermarket staff, the I/O manager will print for staff to view. Recipes stored will include their name, ingredients contained and instructions on how to make
rejected.txt	Stores rejected recipes along with the reason for rejection by logic manager. The 2 main reasons are ingredients not found in recipe list or recipe name

Exception Handling Matrix

Scenarios	Possible Cause	Detection Method	Handling Strategy	User Outcome
Malformed AI output	Bad User inputs or AI Error	Try and catch blocks	Functions raise value errors and try, except blocks	User is informed of the error and prompted to try again until output Is fixed
API network failure	Network connection unavailable	Request timeout	Terminates safely and notifies the user	User is notified of the unavailable network

Missing API key	apikey.env missing or in the wrong folder, or groq API key empty	ai_manager checks environment variable after load_dotenv	Returns a clear error before making API call	User is informed of the error and told to add API key in apikey.env
Missing AI dependencies	groq or python-dotenv not installed in environment	ImportError caught during import	Notify users that AI dependencies are missing	User is told to install the required dependencies in requirements.txt and retry
Invalid user input or assessment data	Empty required fields, invalid menu choices, incorrect number or date formats, and poor quality for AI assessment	Input checks in io_manager and assessment validation in ai_manager	Prompts users to correct interactive input and reject invalid assessment data before calling API	User sees the error and can correct the input or inventory record
Missing or empty data files	food_inventory.csv, recipes.json or accepted_recipe.json is missing or empty	load_json_files() returns an error, logic_manager checks for empty recipes.json	Notify users that JSON/CSV file is empty and continues	User is instructed to run the AI step first or restore the file
Recipe rejected by business rules	Recipe name is not on the approved list, or fewer than 60% of ingredients match	logic_manager rule checks	Saves the recipe to rejected.txt with a rejection reason	User sees the accepted count and rejection reasons. Rejected recipes are never displayed as available
Failed save of recipe changes	Read only file, permission error, or disk full	OSError handled in io_manager	io_manager catches the error and reports it	User sees “Failed to add/update/remove” message, the file remains unchanged
API rate limit reached	Too many requests or provider usage limits exceeded	access_with_client() checks HTTP status 429	SDK retries eligible failures once, and any remaining 429 produces a specific error message	User is instructed to wait or check Groq limits

AI request too large	Request payload exceeds the provider's limit	access_with_client() checks HTTP status 413	Returns an error without producing an assessment	User is told to reduce the inventory payload and retry
Failure to save AI-generated recipes	Folder permissions, disk full or another write failure	suggest_recipe() catches OSError, ValueError and TypeError	Uses a temporary file and atomic replacement, and returns SAVE_ERROR on failure	User sees "Could not save recipe.json. Check folder permissions and retry."